

# GSIP — Build Brief & Handoff

**Grounded Simulation-in-the-loop Platform** — a general, domain-agnostic decision engine. *You point it at any domain by authoring a pack. It frames the problem, refuses to run on made-up inputs, produces numbers only from deterministic code, finds the best trade-offs, and tells you honestly how much to trust the result.*

---

## 0. Read this first (instructions to the assistant receiving this)

You are picking up a project mid-flight. Before you write a single line:

1. **Inventory every file you've been given.** There is an existing repo (the v1 optimiser prototype), two design PDFs (the original dissertation proposal and the "v2 grounded architecture" spec), and possibly more. Read them. Map what already exists against this brief.
2. **Build on what exists — do not rebuild it.** The owner's hard rule: preserve existing work, change only what's required. If v1 code already implements the optimiser, wrap it, don't replace it. Before restructuring anything, show a short gap analysis (what exists / what's missing / what you'll touch) and get a nod.
3. **Hold this line the whole way through — it is the single most important instruction in this document:**

**Reduce the complexity of *building* it. Never reduce the *ambition* of the design.** Stage the work, swap heavy infrastructure for lighter equivalents *that upgrade later without redesign* (pgvector now, Milvus later), and defer polish — but never amputate a load-bearing idea. The provenance discipline, the honest-confidence output, the explore/exploit search, the deterministic core, and — above all — the **domain-general**ity are the point. They stay.
4. **This is a product, not a dissertation.** Ignore any grading/benchmark framing. Optimise for a real, launchable, general platform.
5. **THE CENTRAL AIM — do not lose this:** GSIP is **domain-general**. The platform itself contains **zero** domain knowledge. All domain expertise lives in **packs that users author**. Any bundled pack is only a *test fixture / demo*, never the target. If you ever find yourself specialising the core toward one field, you've broken the project.

---

## 1. What we're building (the pitch)

A general engine for hard decision and design problems — in *any* field. Someone brings a problem (their own domain, via their own pack). The system:

- **frames** it — what kind of problem is this, what actually drives the answer;
- **checks its facts** — every number that matters is traced to a real source, or the system refuses to pretend;
- **simulates** candidate answers with the pack's deterministic code (never an LLM guessing numbers);
- **searches** for the best trade-offs with a sample-efficient optimiser;
- **explains** the result *inside the confidence the evidence supports* — flagging grounded vs guessed, and labelling illustrative results as illustrative.

The same core handles air-quality policy, an electromechanical design, a supply chain, a portfolio, a treatment schedule — because the core knows *how to solve*, and each **pack** supplies *what the domain is*.

**The 30-second "oh, damn" moment:** a user runs a problem in their own domain and, instead of a slick confident number, the system says: *"the input driving your whole result is an assumption nothing backs up — here's the real uncertainty, and I won't dress this up as a prediction."* Every other AI tool confidently makes things up. This one refuses to, and

says so — **for whatever domain you give it**. That refusal, generalised across domains, is the product.

---

## 2. The core idea in plain terms

Two layers, and you need both:

- **v1 is a very good general calculator for hard trade-off problems.** Give it competing goals — cheap *and* safe, fast *and* accurate — and it finds the best mixes using as few expensive simulation runs as possible. It is smart about *searching*. It is dumb about *where the numbers came from*; it trusts whatever it's fed.
- **v2 is v1 plus a bullshit-detector on the inputs.** The failure it kills: an AI invents a number, states it with total confidence, and the calculator computes a flawless answer on top of a fabricated fact — garbage in, *confident* garbage out. v2 checks every decision-critical input before running: real source, or invented? If a make-or-break value is invented, it blocks and tells you, instead of handing over a polished lie.

One line: **v1 makes sure the maths is right. v2 makes sure the maths isn't being done on fake ingredients — for any domain.**

They are not two versions of one thing. **v1 is the engine; v2 is the trust-and-framing shell around it.** You are building the whole vehicle, and it is a *general-purpose* one.

---

## 3. The heart of the project: the domain-pack model + pack authoring

This is what makes GSIP general, and it's the headline capability — not an afterthought.

**The platform is empty of domain knowledge on purpose.** Everything domain-specific is a **pack**. The core just orchestrates the pipeline over whatever packs are registered. This is faithful to the v2 spec, whose worked examples are deliberately from unrelated fields (air-pollution dispersion *and* an electromechanical generator) precisely to prove the core holds no domain expertise.

**Users author their own packs.** A pack is a plugin satisfying a fixed contract:

```
DomainPackBase
├─ state_schema()    → Pydantic model for initial conditions
├─ action_schema()   → Pydantic model for actions / levers
├─ simulate()        → deterministic code → OutcomeBundle (numbers +
uncertainty)
├─ score()           → deterministic metrics from outcome
├─ feasibility()     → is this state+action valid?
├─ cost_model()      → compute cost per fidelity level
├─ fidelity          → DomainPackFidelity (tier · validation · bounds ·
limits · reference)
```

A pack ships alongside one or more **playbooks** (the reusable method for that problem class: required factors, characteristic questions, candidate models, sensitivity priors, known pitfalls, applicability bounds), authored **LLM-drafts / human-ratifies / versioned**.

**The authoring experience is a first-class feature:**

- **A pack SDK + plugin system:** packs are discovered/registered without touching the core (entry-point plugins or a registered interface).
- **A scaffolding tool** (`(gsip new-pack)`): templates + a validator that checks the pack satisfies the contract *and* declares its fidelity honestly (a pack can't ship without stating what kind of model it is).

- **LLM-assisted drafting** (optional but compelling): the user describes a domain in prose; the system drafts a pack skeleton + a playbook; the human edits, ratifies, and versions it. This applies v2's "LLM proposes, human ratifies" rule to authoring itself — the LLM never gets to silently own the framing knowledge.
- A **pack registry**: local, versioned, and shareable, so authored packs can be reused and distributed.
- **Sandboxed execution of user packs** (they run arbitrary code): isolate in a subprocess/container with resource + time limits. Non-negotiable safety requirement, since packs come from users.

**Bundled reference packs exist only to test and demo the contract** — keep two deliberately-unrelated *toy* packs (e.g. a toy dispersion pack and a toy generator pack, mirroring the v2 doc), both explicitly `(fidelity_tier = TOY)`, used to prove genericity, exercise the pipeline, and serve as authoring templates. They are fixtures, never "the domain."

---

## 4. Everything good from v1 — MUST be preserved

Non-negotiable, carried straight through:

- **Numbers-from-code-only.** The LLM frames, routes, proposes, explains. It **never emits a result quantity**. All outcome numbers come from a pack's deterministic `simulate()`.
- **Hybrid cooperating optimiser.** Bayesian Optimisation (sample-efficient) + NSGA-II (multi-objective Pareto fronts) sharing one evaluation pool: BO injects acquisition-maximising candidates into the population; NSGA-II feeds diverse seeds to the surrogate.
- **Parallel evaluation** via Ray (sync + async; reconcile out-of-order results into a shared archive).
- **Reproducibility.** Same run spec + seed → same scenarios → same numbers.
- **Deterministic scoring.** Pure math rubrics; no AI in scoring.
- **Audit / run ledger.** Every decision, assumption, and value provenance logged; the whole run replayable.
- **Generic platform / domain-pack split.** *This is now the spine of the whole product* — the engine is domain-agnostic; all domain knowledge lives in packs.
- **Config-as-code + containerised.** YAML run specs, Docker, single-command reproducible runs.
- *(Keep a ZDT/DTLZ benchmark harness as internal optimiser validation — it proves the engine works, domain-independently.)*

---

## 5. Everything from v2 — MUST be present

The grounding + framing shell. This is the differentiator; don't water it down:

- **Provenance model (core innovation).** Every value is a `ParameterValue` carrying origin + uncertainty, never a bare number. Taxonomy: `USER_SUPPLIED`, `MEASURED`, `RETRIEVED`, `COMPUTED` (inherits weakest parent), `DEFAULT` (ungrounded, declared), `LLM_RECALL` (ungrounded, unverified — the floor). `COMPUTED`-inherits-weakest-parent stops laundering a guess into a grounded-looking number via arithmetic.
- **Provenance gate.** Criticality (how much a parameter moves the answer) × provenance (how well-grounded) → `RUN` / `FLAG` / `BLOCK`. **The one rule that matters: a decision-critical parameter may never silently take an LLM-recalled value.** Ground it or refuse. Override is explicit, logged, and stamps the whole result "ungrounded/illustrative."

- **Problem classifier.** Multi-label, calibrated confidence, and — critically — **abstention** ("I have no validated frame for this"). Abstention is a first-class output. Routes to whatever packs/playbooks are registered; frame-disambiguation when frames conflict.
- **Playbooks.** Per problem-class curated method (fields above). LLM-drafts / human-ratifies / versioned — never LLM-authored-unchecked.
- **Parameter triage.** Morris elementary-effects screening on the cheapest-fidelity model → rank each parameter `[CRITICAL / MODERATE / LOW]`. Replaces LLM-asserted importance with *measured* importance. Sets per-parameter gate strictness and orders clarification.
- **Triage-driven clarification & grounding.** Ask first (and sometimes only) about parameters both decision-critical and ungrounded. Each question declares the provenance it produces. The system never satisfies a critical parameter by letting the LLM fill it in.
- **Domain packs carry a fidelity descriptor.** `[fidelity_tier]` (TOY / REDUCED\_ORDER / VALIDATED / CALIBRATED) + `[validation_status]` + `[applicability_bounds]` + `[known_limitations]` + `[reference]`. A toy model can never read as validated; fidelity travels *with* the numbers into the report.
- **Split-budget scenario generation.** Exploit (~60%, grounded AI-seeded, realistic) + Explore (~40%, space-filling LHS/Sobol, *un-fenced* by AI priors so novel optima aren't suppressed). Any specific value in an AI-proposed scenario is `[LLM_RECALL]` and still passes the gate — no exemption.
- **Uncertainty-aware outcomes.** `[simulate()]` returns metrics **and** uncertainty; the Pareto front carries uncertainty so "best" is never a false-precise point. Cheap output-uncertainty via triage sensitivities × input uncertainty (no separate Monte-Carlo pass).
- **Confidence-calibrated explanation.** LLM writes the report, bound by two hard rules: (1) fidelity propagation — illustrative results are *called* illustrative; (2) provenance disclosure — surfaces which critical inputs were grounded vs defaulted vs overridden, leading with the caveat if it ran on an overridden ungrounded input.
- **Run ledger + orchestration.** Append-only audit of every stage; workflow with a human-in-the-loop pause at the gate and at clarification.

---

## 6. Architecture — the full pipeline (nothing amputated)

The ten-stage flow, engine inside shell, generic over packs:

- 1 Intake – natural-language problem (any domain)
- 2 Problem classification – multi-label · calibrated · can ABSTAIN
- 3 Playbook retrieval – required factors · characteristic questions · candidate packs
- 4 Parameter triage – Morris screening → CRITICAL/MODERATE/LOW
- 5 Grounding & clarification – resolve each value, tag provenance (triage-ordered)
- 6 PROVENANCE GATE – criticality × provenance → RUN / FLAG / BLOCK  
← the trust wall
- 7 Scenario generation – Exploit (grounded AI seeds) + Explore (un-fenced LHS/Sobol)
- 8 Simulation – the USER'S pack · deterministic · fidelity descriptor
- 9 Scoring + optimisation – deterministic scoring · hybrid BO + NSGA-II  
← the v1 engine
- 10 Confidence report – caveated by provenance + model fidelity  
    ↳ Run ledger appends every decision, assumption & provenance tag throughout

**Zones:** Input → *Framing* (LLM frames, never asserts fact) → **Trust wall** (gate enforces truth) → *Deterministic core* (numbers from the pack's code). Keep these boundaries crisp in the layout.

**Service layout** (adapt to the existing repo; don't fight it):

```
core/          provenance.py (ParameterValue, gate()), classification.py,
playbook.py,
              triage.py, clarification.py, fidelity.py, outcomes.py,
ledger.py
engine/        optimiser (BO + NSGA-II hybrid), scenario gen
(exexploit/explore), Ray runner
packs/         base.py (DomainPackBase + fidelity contract)
              sdk/          scaffolding (gsip new-pack), validator, plugin
discovery, sandbox
              examples/    toy_dispersion/, toy_generator/    (TOY
fixtures only)
activities/    classify · retrieve_playbook · triage · ground_and_clarify
· gate ·
              propose_scenarios · expand_scenarios · simulate · optimise
· explain
workflow/      run orchestration + human-in-loop pause at
gate/clarification
api/           POST /runs · POST /runs/{id}/clarify · POST
/runs/{id}/override · pack CRUD
web/           the honest-output UI + the pack-authoring UI
registry/      versioned, human-ratified playbooks + authored packs
```

**Data structures:** use the Pydantic models from the v2 architecture PDF verbatim as the starting contract (`Provenance`, `ParameterValue`, `ProblemClassification`, `Playbook`, `Criticality`, `TriageResult`, `ClarificationQuestion`, `DomainPackFidelity`, `GateVerdict`, `ProvenanceGateDecision`, `EnrichedState`, `OutcomeBundle`, `ConfidenceReport`, and `gate()`). Implement them; don't reinvent.

## 7. Tech stack + the complexity-reduction swaps

Core is Python (optimiser, provenance, pack contract are Python-native). The UI is where the product is felt.

| Concern          | Launch choice (lower build cost)                                                                                                                                                                 | Upgrade later (no redesign)      | Why the swap keeps ambition intact                 |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|----------------------------------------------------|
| Optimiser        | pymoo (NSGA-II) + BoTorch (BO), existing v1 code                                                                                                                                                 | more backends (NSGA-III, MOEA/D) | pluggable behind one interface                     |
| Parallelism      | Ray, single-node                                                                                                                                                                                 | Ray cluster / cloud              | identical API local→cluster                        |
| Vector/retrieval | <b>pgvector</b> on Postgres                                                                                                                                                                      | <b>Milvus</b>                    | same retrieval port                                |
| Classifier       | <b>prompted LLM + calibration</b>                                                                                                                                                                | fine-tuned SLM                   | same <code>classify()</code> + abstention contract |
| Pack execution   | subprocess sandbox + resource limits                                                                                                                                                             | gVisor/container-per-pack        | same pack interface; isolation hardens later       |
| Orchestration    | Temporal (Python SDK) — worth it early for the human-in-loop gate pause; a persisted async state machine ( <code>awaiting_clarification</code> state) is an acceptable <i>temporary</i> stand-in | full Temporal                    | the gate pause is a design requirement either way  |
| Backend API      | FastAPI                                                                                                                                                                                          | —                                | —                                                  |
| DB / ledger      | Postgres (SQLAlchemy + Alembic)                                                                                                                                                                  | —                                | append-only ledger + provenance + pack registry    |
| Frontend         | React + TypeScript                                                                                                                                                                               | —                                | honest-output screen + pack-authoring screen       |
| LLM calls        | one adapter (Anthropic/OpenAI)                                                                                                                                                                   | swap freely                      | model-agnostic                                     |
| Packaging        | Docker + docker-compose, YAML run specs                                                                                                                                                          | —                                | reproducible, single-command                       |

**Rule for every swap:** it must sit behind a stable interface so the "later" version drops in without touching callers. If a shortcut would force a redesign to undo, it's not allowed — take the harder path now.

### 8. Phased build plan (doable, staged, nothing dropped)

Each phase ends with a working, testable slice. Do not start a phase before the previous is green.

**Phase 0 — Ground truth & scaffolding.** Inventory existing repo + PDFs. Gap analysis. Stand up the mono-repo, Docker, Postgres, FastAPI skeleton, CI. Port/confirm the v1 optimiser runs (validate against a ZDT/DTLZ problem to prove the engine, domain-independently).

**Phase 1 — The spine: provenance + gate (build first, always).** `ParameterValue`, provenance taxonomy, `COMPUTED`-inherits-weakest-parent, `gate()`. Run ledger logs every value + gate decision. Unit tests for the full gating matrix. Everything downstream depends on this.

**Phase 2 — The pack contract + SDK (this is the general core).** `DomainPackBase` with the fidelity descriptor; the plugin/discovery mechanism; the pack validator (contract + honest-fidelity check); sandboxed pack execution. Ship the two **toy** reference packs (dispersion + generator) purely to exercise the contract and prove genericity. Uncertainty-aware `OutcomeBundle`.

**Phase 3 — Optimiser integration (the v1 engine, inside).** Hybrid BO + NSGA-II cooperating loop over *any* registered pack; split-budget scenario generation (exploit grounded seeds + explore un-fenced LHS/Sobol); Ray parallel eval; uncertainty carried onto the Pareto front. Scenario values pass the gate.

**Phase 4 — Triage + grounding + clarification.** Morris screening → criticality; triage-driven clarification (critical-ungrounded first); questions declare produced provenance; grounding resolves values with real provenance. Triage sensitivities → per-parameter gate strictness → cheap output-uncertainty bands.

**Phase 5 — Classifier + playbooks + registry.** Prompted-LLM multi-label classifier with calibration + abstention + frame-disambiguation; playbook schema + versioned registry; routing across *whatever packs are registered*. One playbook per reference pack (LLM-drafted, human-ratified).

**Phase 6 — Orchestration + human-in-loop.** Run workflow with the gate/clarification pause; `POST /runs`, `/clarify`, `/override` (explicit, logged); resume-after-clarification.

**Phase 7 — Confidence report + honest-output UI.** LLM report bound by fidelity propagation + provenance disclosure. The React screen showing grounded-vs-guessed, uncertainty bands, and the Pareto trade-offs — generic over domain. This screen is the demo.

**Phase 8 — Pack-authoring experience (headline feature, general aim).** `gsip new-pack` scaffolding; the pack-authoring UI; **LLM-assisted pack + playbook drafting** (describe a domain → draft skeleton → human ratifies/versions); pack registry sharing; hardened sandboxing. *This is what delivers "users author their own packs" — treat it as core, not polish.*

**Phase 9 — Upgrades (non-blocking).** pgvector→Milvus; prompted-LLM→fine-tuned SLM; calibration hooks where ground truth exists (compare sim vs reality, upgrade `validation_status`); more optimiser backends.

**Definition of done for launch:** Phases 0–8. A user can register/author a pack in a domain *of their choosing*, pose a problem, watch the gate visibly block/flag ungrounded critical inputs, get an uncertainty-aware Pareto front from the pack's deterministic sims, and read a report that never upgrades an illustrative result to a prediction.

---

## 9. Honest limitations to carry forward (don't overclaim)

From the v2 spec's §18 — state plainly, don't paper over: reliable open-set abstention is an unsolved research problem and the riskiest component; playbook + pack curation is real human cost and goes stale; retrieval makes a value *attributable*, not *true*; Morris screening still costs something on expensive sims; fidelity tiers are coarse honest-

signalling, not error bounds; ground-truth calibration only exists for some problems; and running user-authored packs is an ongoing security surface. Credibility comes from admitting these, not hiding them.

---

## 10. The Cursor prompt (paste this into Cursor)

Copy the block below into Cursor's Composer/agent. Feed this brief + the two PDFs + the existing repo into the workspace first.

---

### MASTER PROMPT — GSIP (Grounded Simulation-in-the-loop Platform), a general domain-agnostic engine

You are building GSIP: a **general** decision platform where an LLM frames a plain-English problem but is *architecturally forbidden* from supplying the facts that decide the answer. Numbers come only from deterministic simulation code inside a **domain pack**; a provenance gate refuses to run when a decision-critical input is backed only by model memory; results are reported inside the confidence the evidence supports. **The platform contains no domain knowledge — all domain expertise lives in packs that users author.** Full spec is in `GSIP_Build_Brief_and_Cursor_Handoff.md` and the two design PDFs in the workspace — treat those as source of truth.

#### Absolute rules:

1. Read the existing repo and the PDFs first. Produce a short gap analysis (exists / missing / will-touch) before generating code. Build on existing code; do not rebuild working parts.
2. **Reduce build complexity, never design ambition.** Use the launch-stack swaps in §7 (pgvector-before-Milvus, prompted-LLM-before-SLM, etc.), but only behind stable interfaces so upgrades need no redesign. Never drop a load-bearing idea.
3. **The platform is domain-general. Never specialise the core toward any single field.** Any bundled pack is a test fixture only. Domain knowledge belongs exclusively in user-authored packs.
4. The LLM never emits a result number. All outcomes come from a pack's `simulate()`.
5. Work **one phase at a time** (Phases 0-9 in §8). After each phase: run tests, show what's green, and stop for confirmation before the next. Do not scaffold everything at once.

**Stack:** Python 3.12, FastAPI, Pydantic (exact models from the v2 PDF), SQLAlchemy+Alembic on Postgres (append-only ledger + provenance + pack registry), Ray (parallel eval), pymoo + BoTorch (NSGA-II + BO hybrid), pgvector (retrieval), sandboxed subprocess pack execution, Temporal Python SDK (workflow + human-in-loop gate pause; a persisted async state machine is an acceptable temporary stand-in if Temporal blocks progress), React + TypeScript frontend, one LLM adapter, Docker + docker-compose, YAML run specs.

**Start now with Phase 0 only:** inventory, gap analysis, mono-repo + Docker + Postgres + FastAPI skeleton + CI, and confirm the v1 optimiser runs against one ZDT/DTLZ problem to prove the engine. Then stop and report before Phase 1.

**Phase 1 (do not start until told):** the provenance spine — `ParameterValue`, provenance taxonomy, `COMPUTED` inherits weakest parent, `gate()`, run-ledger logging, and unit tests covering the entire criticality×provenance gating matrix.

Then proceed phase by phase through §8. Crucially, **Phase 2 (the domain-pack contract + SDK) and Phase 8 (the pack-authoring experience) are what make this general — treat them as core deliverables, not optional extras.** For every phase, deliver a working, tested slice with acceptance criteria met, and pause for review.



---

## 11. One-paragraph version (if you need to explain it fast)

GSIP is a general engine for hard trade-off problems in *any* domain. You describe a problem; it figures out what actually drives the answer, and before it computes anything it checks whether the make-or-break inputs come from real sources or are just the AI's guesses. If a critical input is invented, it refuses to run and says so, instead of producing a confident, false-precise answer. When it does run, a deterministic simulator — supplied by a **pack you author for your domain**, never the AI — produces the numbers, a sample-efficient optimiser finds the best trade-offs, and the result is reported honestly, with uncertainty bands and a clear label of what was grounded versus assumed. The optimiser is powerful (the "v1" engine: hybrid Bayesian + evolutionary search, parallelised); the grounding discipline around it (the "v2" shell: provenance, the gate, honest confidence) is what makes it trustworthy; and the domain-pack model — a fixed contract plus an authoring toolkit — is what makes it work for *everyone's* domain, not just one.